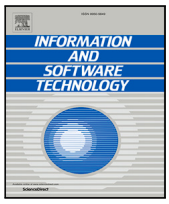
Information and Software Technology 195 (2026) 108117

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof



An empirically-driven clustering framework for NoSQL data warehouse conversion: Optimizing column family design from relational big data using HDBSCAN

Mohamed Mouhiha *, Abdelfettah Mabrouk

Department of Computer Science, Chouaïb Doukkali University, ESTSB, ELITES Lab, El Jadida, Morocco

ARTICLE INFO

Keywords:
Clustering
NoSQL
Datawarehouse
NoSQL datawarehouse
Hbase
Optimization
HDBSCAN
DBSCAN
ISODATA
OPTICS
Xmeans

ABSTRACT

Context: Modern businesses increasingly rely on integrating external data sources, particularly Linked Open Data, to improve decision-making processes. Traditional relational data warehouses struggle to handle diverse Big Data sources effectively, creating a need for more flexible storage solutions.

Objectives: This study aims to develop a comprehensive framework for converting traditional relational data warehouses to NoSQL column-oriented systems. We specifically address two major limitations in existing approaches: their focus on only query-relevant columns and the manual requirement for specifying cluster numbers.

Methods: We implemented and compared five clustering algorithms — Xmeans, DBSCAN, OPTICS, HDBSCAN, and ISODATA — to optimize column family structures during the conversion process. Our framework considers all warehouse attributes rather than just query-specific ones, and automatically determines the optimal number of clusters without manual intervention. We conducted extensive experimental testing to evaluate performance against existing methods, including the CLARANS algorithm.

Results: HDBSCAN demonstrated superior performance compared to all other clustering methods tested, including CLARANS which was previously considered the leading approach. The experimental results showed significant improvements in query response times and better adaptation to complex data relationships. Our framework successfully addressed the limitations of previous conversion methods by incorporating comprehensive attribute analysis.

Conclusion: The proposed clustering framework offers a more effective solution for relational-to-NoSQL data warehouse conversion. By automatically optimizing cluster configurations and considering all warehouse properties, this approach provides better performance and flexibility compared to existing methodologies, making it particularly valuable for organizations dealing with heterogeneous Big Data sources.

1. Introduction

Data warehousing has served as a cornerstone of organizational analytics for decades, providing structured environments for managing one of an organization's most critical assets: its data [1]. As the digital landscape evolves, the emergence of big data has significantly challenged traditional data warehouse ecosystems, prompting a reevaluation of established architectures [2]. While some analysts initially predicted the obsolescence of traditional data warehouses in favor of pure big data systems, the prevailing consensus among data warehousing communities supports extending current frameworks to accommodate big data requirements rather than wholesale replacement [3]. This extension necessarily involves integrating the fundamental pillars of big data — commonly characterized as the 5Vs: Volume, Variety, Velocity, Value, and Veracity — throughout the decision-making process [4]. Traditional data warehouses frequently encounter limitations when required to incorporate external data sources, which ultimately restricts organizations from extracting meaningful insights and contradicts the primary objectives of decision support systems [5]. To effectively support managerial decision-making, organizations must enhance their existing data warehouses by incorporating external big data relevant to their operations [6]. However, conventional data warehouses built on relational database technology are fundamentally unsuited for handling external big data characterized by high volume and heterogeneous data formats [7]. These challenges necessitate schema-less distributed systems capable of efficiently accommodating both internal and external data sources [8]. This technological evolution has catalyzed

* Corresponding author.
E-mail address: mouhihamohamed@gmail.com (M. Mouhiha).

https://doi.org/10.1016/j.infsof.2026.108117
Received 15 June 2025; Received in revised form 20 December 2025; Accepted 9 March 2026
Available online 10 March 2026
0950-5849/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

research interest in revitalizing data warehouse architectures to meet contemporary analytical demands. It is important to distinguish between two related but distinct research areas: general migration from relational databases to column-oriented systems, which must handle diverse schemas and mixed workloads across various application contexts, and the more specialized case of data warehouse conversion, which benefits from the structured nature of warehouse schemas with their predictable fact-dimension relationships. Current research approaches documented in the literature have primarily focused on direct conversion methodologies from classical relational data warehouses to column-oriented NoSQL data warehouses [9]. More advanced efforts have employed clustering algorithms to optimize this conversion process, though these approaches remain limited and exhibit several significant shortcomings [10]. As a result, the challenge of designing truly optimized column-oriented NoSQL data warehouses remains largely unresolved. Existing clustering-based approaches, including K-means and K-medoids, present two major limitations: first, they typically consider only query-accessed attributes rather than the complete attribute set from the traditional data warehouse; second, they require pre-specification of the cluster count without providing optimization mechanisms for this critical parameter [11,12]. While CLARANS has demonstrated superior results compared to K-means and K-medoids when using the complete attribute set, further investigation into alternative clustering methodologies is warranted. This paper aims to address these limitations by exploring additional clustering techniques — namely Xmeans, DBSCAN, OPTICS, HDBSCAN, and ISODATA — for modeling optimal column family structures in NoSQL data warehouses. Our approach seeks to overcome existing constraints by evaluating these algorithms' effectiveness in considering the complete attribute set while providing mechanisms to optimize cluster configurations without predetermined specifications. Through comparative analysis, we aim to advance the state of the art in column-oriented NoSQL data warehouse design to better fulfill the evolving analytical requirements of modern organizations.

The structure of the remainder of this paper is outlined as follows: Section 2 presents the problem statement and demonstrates the challenges through migration results. Section 3 delves into related optimized and unoptimized approaches. Section 4 describes the clustering techniques and target system. Section 5 presents our methodology. Section 6 details the implementation and experimental results. Section 7 concludes our investigation with a comparison and discussion.

2. Problem statement

When building data warehouses, column-oriented NoSQL databases offer a different approach from traditional databases. These systems store data in a unique way that requires special design thinking to use them effectively. In these databases, all data goes into a single large table with one or more column families. Each column family holds a group of related attributes, creating a vertical split of the data. Fig. 1 shows how this works with a table T that has two column families. Every column family CF_i contains several attributes with their values. Each row of data has a unique Row-key (R_i). An important feature is that all data with the same Row-key is stored together. The column family name helps identify its columns, while the Row-key acts as the main key for all attributes. Query speed in this setup depends mainly on two things: which columns you choose and which rows you need. For example, if a query needs attributes (Att1, Att2, Att3) that all live in CF_1, the system only needs to search in that one place. But if a query needs attributes (Att1, Att4) from different column families, the system must search in both CF_1 and CF_2, scanning all values in each. This makes the query slower. Traditional data warehouses moving to columnar NoSQL systems typically rely on denormalization strategies. While some earlier studies explored clustering approaches like K-means, K-medoids, and CLARANS to organize attributes into column families optimized for specific query patterns, these techniques have notable limitations. They require users to predetermine the number of column families without offering guidance on optimal configuration. Additionally, these methods only consider attributes that appear in query SELECT and WHERE clauses, ignoring other available attributes in the data warehouse schema. This selective approach can be problematic because when queries span multiple column families, it creates management overhead and undermines the performance benefits that column family grouping is supposed to provide. To illustrate the real impact of these limitations, consider what happens when we migrate a traditional data warehouse to a column-oriented NoSQL system. In our experimental evaluation using the TPC-DS benchmark with approximately 29 million records, we tested different clustering approaches to organize attributes into column families. The results show just how much the choice of method matters. When using traditional K-means clustering, simple queries took 38.7 s to execute, while complex queries required 78.1 s. K-medoids performed slightly better at 35.4 s for simple queries and 72.8 s for complex ones. Even CLARANS, which is considered one of the better existing methods, needed 32.1 s for simple queries and 65.5 s for complex queries. These performance differences directly stem from how well the column families are organized. When attributes that are frequently accessed together end up in different column families, the system has to scan multiple storage locations for each query, which slows everything down. The problem becomes even clearer when we look at what happens with poorly designed column families. If a query needs attributes spread across many column families, the system must access each family separately, multiplying the work required. This is why getting the column family design right is so critical. The challenge is that existing methods do not give us good ways to automatically figure out the best way to group attributes. They either make us guess how many column families to create, or they only look at a subset of the attributes, which means the design might work for current queries but break down when new queries come along that use different attributes. To solve these problems and better balance the load across column families, we need a better design approach for data warehouses on column-oriented NoSQL databases. We propose such an approach in the next section.

3. Related work

When it comes to making data warehouses work better, researchers have tried many different things. Some work focuses on making queries run faster, others on improving how data gets loaded and processed. For example, people have used AI to automate ETL processes and help with real-time analytics [13]. Others have built scheduling systems that cut query times significantly [14], or used caching to speed up data access [13]. There is also work on optimizing ETL processes in the cloud, like combining different algorithms to reduce storage needs and improve performance [15]. Companies like Microsoft have also worked on improving query optimizers for cloud data warehouses [16]. But here's the thing: most of this work focuses on making queries run faster or improving how data flows through the system. What is missing is work on the actual design of column families in NoSQL data warehouses. That is what we focus on in this paper. The way you organize data into column families has a huge impact on how well everything works. In the sections below, we look at what others have done and where the gaps are.

3.1. Classical approaches

The first group of papers we look at deals with converting data warehouses to NoSQL systems, but they do not use any clustering algorithms. Instead, they test different ways of organizing the data and see what works best.

One early study [17] tried three different ways to put data into column-oriented NoSQL systems. They tested putting everything together, separating facts and dimensions, and spreading data across

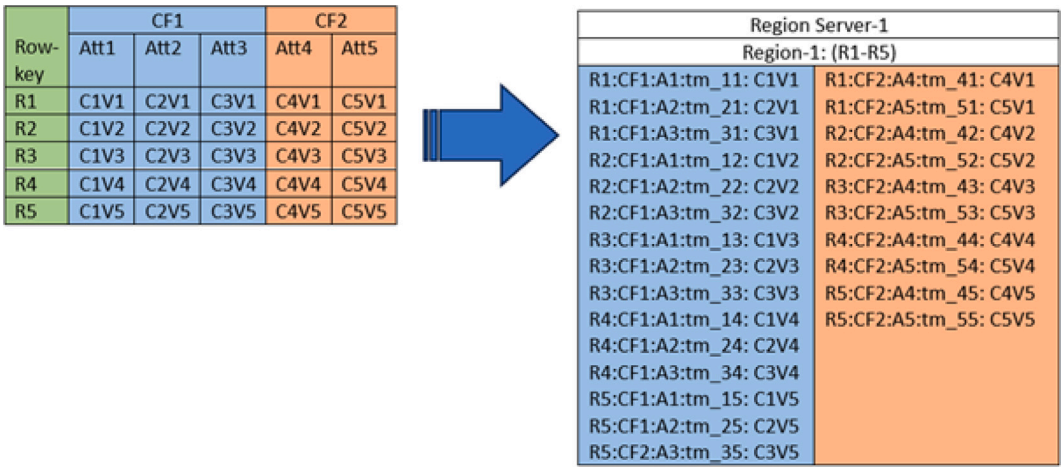


Fig. 1. How data is stored in column-oriented NoSQL databases.

multiple tables. What they found was interesting: the approach that saved the most storage space actually made queries slower because it needed more joins. The simpler approaches worked about the same. This shows there is always a trade-off between saving space and making queries fast.

Other researchers looked at whether column-oriented or document-oriented databases work better for data warehouses [18]. They found that column systems like Cassandra load data much faster, but document systems like MongoDB handle complex queries better when you need lots of attributes at once. This makes sense because in document systems, all the data for one record sits together, while column systems spread it out.

When researchers dug deeper into column-oriented systems specifically, they tested different ways to structure the data [19]. They tried normalized structures, denormalized structures, and denormalized structures with column families. With a huge dataset of 6 billion records, the denormalized approaches ran queries up to three times faster. The version with column families worked especially well for queries that focused on one dimension. This tells us that denormalization helps a lot, and that how you group attributes into column families matters.

More recent work looked at specific ways to design data warehouses in HBase [20]. They tested three designs: putting everything in one column family, putting each dimension in its own family, and combining the fact table with the date dimension. Each design worked best for different types of queries. The single-family approach worked well when queries needed many dimensions, but struggled with simpler queries. The separate-families approach worked for low-dimensional queries. The combined approach worked great for date-related queries. This shows that the best design depends on what kinds of queries you run most often.

To help people actually do these conversions, some researchers built frameworks with clear rules [21]. One framework called NOSOLAP gives step-by-step instructions for converting traditional data warehouses to Cassandra. It tells you how to turn each part of your warehouse model into Cassandra tables and columns. This kind of framework makes it easier for people to actually do the conversion, but it still does not automatically figure out the best way to organize column families.

What all these papers have in common is that they test different structures manually. They do not use algorithms to automatically figure out the best way to group attributes. They also do not consider all the attributes in the warehouse—they mostly focus on the ones that queries actually use. This is where the next group of papers comes in.

3.2. Optimized approaches

The second group of papers uses clustering algorithms to automatically figure out how to organize column families. This is a step forward, but these methods still have some problems.

The first paper in this group [22] used k-means clustering to group attributes into column families. They built a tool that extracts all attributes, uses k-means to group them, creates the schema, and loads the data. When they tested it, their approach with 11 column families cut query time by about 19% compared to having one family per table, and by 34% compared to putting everything in one family. This shows that clustering can help, but they had to pick the number 11 themselves. They also found that you cannot just rely on statistics to pick the right number—you need to actually test different options.

Other researchers tried using k-medoids instead of k-means [23]. They wanted to make complex queries faster by reducing how much data the system has to look through. Their method first groups similar queries together using k-medoids, then uses a particle swarm algorithm to create the column families. They tested it on HBase and found it handled large datasets better and ran queries faster, especially when the workload changed over time. But like the k-means approach, they still had to decide how many clusters to use beforehand.

Some researchers combined statistical analysis with different algorithms [24]. They looked at query patterns to find which attributes get used together, then used either Particle Swarm Optimization or k-means to create column families. They tested four approaches and found their optimization method worked best with 8–11 column families. But again, they only looked at attributes that appeared in queries, and they had to pick the number of families manually.

The most advanced work in this group uses CLARANS clustering [25]. This is better because it solves two problems at once: it automatically figures out how many column families to create, and it includes all attributes from the warehouse, not just the ones in queries. When they tested it, CLARANS reduced 175 column families down to 45 with zero error. The downside is it takes more time to run than the other methods. But it shows that you can automatically optimize the number of families and include all attributes.

So what is the pattern here? The clustering-based approaches are better than the manual ones, but most of them still have two big limitations. First, they only look at attributes that queries actually use, ignoring the rest. Second, they make you pick how many column families to create, and they do not help you figure out the best number. CLARANS fixes both of these problems, but there might be other clustering algorithms that work even better. That is what we explore in this paper. While CLARANS represents an important

step forward in automatically determining column family counts, our proposed approach differs fundamentally in the clustering technique it employs. CLARANS is a partitioning-based algorithm that works by selecting representative objects (medoids) and iteratively refining cluster assignments. In contrast, our method uses density-based clustering algorithms, specifically HDBSCAN, which identifies clusters based on the density of data points rather than predefined partitions. This difference matters because density-based methods can discover clusters of varying shapes and sizes without assuming that all clusters have similar characteristics. More importantly, density-based approaches can identify noise points and outliers that do not belong to any cluster, which partitioning methods like CLARANS must assign to some cluster. For column family design, this means our approach can better handle attributes that have weak relationships with others, potentially leaving them unassigned or creating more focused column families. Additionally, while CLARANS automatically finds an optimal number of clusters, it still operates within a partitioning framework that assumes every data point belongs to exactly one cluster. Our density-based approach offers more flexibility in how attributes are grouped, potentially leading to better column family structures that reflect the actual relationships in the data rather than forcing a partition-based structure.

3.3. General relational database migration to column-oriented systems

Beyond data warehouse conversion, researchers have also worked on migrating regular relational databases to column-oriented NoSQL systems. This is actually a tougher problem than what we are tackling in our paper. For example, one study looked at moving from MySQL to HBase by breaking down the process into three main steps: understanding the source database structure, defining what the target database should look like, and then translating everything using specific rules [26]. Another recent paper improved on this by combining three techniques — denormalization, looking at how data gets accessed, and using nested schemas — to cut down on wasted storage space and speed up queries [27]. Their tests showed they could reduce query times by about 29% compared to older methods. Here's why regular database migration is harder: data warehouses have a predictable structure with fact tables and dimension tables that follow clear patterns. Regular databases do not. They can be organized in all sorts of ways, with different normalization levels and no standard layout. You cannot assume anything about how the data is structured. That is where our work is different. We are focusing specifically on data warehouse conversion, which lets us take advantage of that predictable structure. We can use the relationships between facts and dimensions to guide our clustering. We also go further than previous work by looking at the entire set of attributes in the warehouse, not just the ones that show up in queries. And we use HDBSCAN, which automatically figures out the right number of column families without anyone having to guess upfront. This combination — using the warehouse structure, considering all attributes, and automating the clustering — is what makes our approach new compared to both the regular database migration work and the earlier warehouse conversion methods.

3.4. Summary of research gaps

Looking at all this work, we can see several gaps that need to be filled. First, most clustering methods only consider attributes that show up in queries. They ignore other attributes in the warehouse, which means the column family design might not work well if new queries come along that use different attributes. Second, most methods require you to manually pick how many column families to create. They do not automatically figure out the best number, so you have to try different options and see what works. Third, there has not been much work exploring density-based clustering algorithms like HDBSCAN, DBSCAN, and OPTICS for this problem. Most papers stick with k-means, k-medoids, or CLARANS. Fourth, we need methods that automatically determine the optimal number of clusters without requiring manual testing.

Our work addresses these gaps by testing multiple clustering algorithms, including density-based ones, that can automatically figure out the best number of column families while considering all attributes in the warehouse, not just the ones in current queries.

4. Clustering techniques and target system

In this section, we describe the clustering methods used to group attributes from the traditional data warehouse, as well as the target NoSQL system where these groups are stored. The main goal is to reorganize the data into column families that are more efficient for storage and querying in a NoSQL environment.

4.1. Clustering techniques

Automatic classification or clustering represents a critical step in Knowledge Extraction from Data processes. It aims to develop optimal partitioning by organizing data into classes that share similar characteristics, where data points typically exist as measurement vectors in a multidimensional space. Intuitively, vectors within a valid cluster demonstrate greater similarity to each other than to vectors in different groups [28]. The fundamental goal of these methods is to identify meaningful groupings from unlabeled data sets that share semantic similarities, enabling users to construct cognitive models that reveal the inherent structure within data. Clustering algorithms generally fall into four main categories: hierarchical algorithms, partitioning algorithms, density-based algorithms, and grid-based quantification methods. For our specific challenge of designing column-family structures in NoSQL data warehouses, partitioning algorithms appear most suitable as they directly divide a set of data (the attributes of traditional relational data warehouses) into k classes (column families). Each class must contain at least one attribute, and each attribute must belong to exactly one class—unlike fuzzy classification approaches which permit overlapping membership. Traditional partitioning algorithms typically follow this general process:

- Determine the number of clusters
- Initialize cluster centers
- Partition the dataset
- Recalculate cluster centers

A persistent challenge when applying partitioning algorithms is determining the optimal number of output classes. These techniques generally produce clusters by optimizing objective functions defined either locally (on data subsets) or globally (across all vectors), ensuring intra-cluster similarity and inter-cluster dissimilarity. For k-class classification, these algorithms generate an initial partition and then iteratively improve it by reassigning elements between classes, which allows recovery from potentially poor initial partitions. K-means and K-medoids have been widely implemented for data warehouse conversion projects. CLARANS (Clustering Large Applications based on RANdomized Search), an extension of K-medoids, has demonstrated superior results in this field [4] by employing a randomized sampling approach that improves efficiency when handling large datasets. Unlike its predecessors, CLARANS explores the search space more effectively by examining a dynamically selected sample of neighboring clusters rather than the entire neighborhood, significantly reducing computational overhead while maintaining clustering quality. However, these approaches suffer from notable limitations. First, they typically focus exclusively on attributes involved in query patterns rather than considering the comprehensive attribute set available in the traditional data warehouse. This selective approach potentially overlooks important structural relationships that could enhance the column-family design. Second, they require analysts to specify the number of clusters

beforehand without providing optimization mechanisms or adaptation techniques to determine the most appropriate configuration. This predetermined parameter can significantly impact the quality of the resulting column families and, consequently, the performance of the NoSQL data warehouse.

Contemporary clustering methodologies beyond traditional partitioning techniques such as K-means, K-medoids, and CLARANS offer promising applications for column-family design optimization. These alternative approaches provide mechanisms to overcome the limitations of predetermined cluster counts and selective attribute consideration that characterize conventional methods. By implementing adaptive parameter selection and comprehensive data relationship analysis, these newer clustering strategies can potentially enhance NoSQL data warehouse performance through more sophisticated partitioning that better reflects the underlying data structures and access patterns. This research specifically examines the application of ISODATA, X-means, HDBSCAN, DBSCAN, and OPTICS algorithms in column-family design contexts.

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise) offers significant advantages by discovering clusters of arbitrary shapes based on density connectivity. Unlike partition-based methods, DBSCAN does not require pre-specifying the number of clusters and can identify outliers as noise. This capacity to determine cluster numbers automatically makes it particularly valuable for CN-DW design where optimal column-family counts may not be known in advance [29].
- HDBSCAN (Hierarchical DBSCAN) further evolves density-based clustering by combining hierarchical approaches with DBSCAN's principles. It constructs a density-based clustering hierarchy and extracts a flat clustering based on cluster stability. This method adapts well to varying densities and produces clusters of higher quality with fewer parameter tuning requirements—a significant advantage when modeling complex data warehouse structures [30].
- OPTICS (Ordering Points To Identify the Clustering Structure) extends DBSCAN's principles by addressing varying density clusters. It creates an augmented ordering of database points to identify structural clusters based on core-distance and reachability measurements. This approach produces a visual representation that facilitates intuitive understanding of cluster structures across different densities, potentially revealing natural groupings of attributes [31].
- ISODATA (Iterative Self-Organizing Data Analysis Technique) enhances k-means through additional operations including splitting and merging clusters based on standard deviations and distances between centers. Its self-adjusting nature makes it adaptable to discovering the natural number of column families without requiring precise initial parameters [32].
- X-means extends the classic K-means algorithm by automatically determining the optimal number of clusters, which is particularly valuable for column-family design in NoSQL data warehouses. Unlike K-means, which requires manually specifying how many clusters to create, X-means starts with a minimum number of clusters and intelligently splits them until finding the best arrangement. It uses a scoring system based on the Bayesian Information Criterion (BIC) to evaluate different cluster configurations and choose the one that best balances complexity and accuracy. This smart approach helps database designers discover natural groupings in their data without needing to guess the right number of column families in advance, potentially leading to more efficient data storage and faster query performance [33].

In our context of designing optimized NoSQL column-oriented data warehouses, these alternative clustering approaches offer promising pathways to overcome the limitations of traditional methods. By automatically determining appropriate cluster numbers and considering the full attribute landscape rather than just query-used attributes, these techniques may produce more efficient and naturally aligned column-family structures that better reflect the inherent relationships between data warehouse attributes.

4.2. Target system: HBase

HBase [34] represents a column-oriented NoSQL database architecture that shares visual similarities with traditional relational databases while operating on fundamentally different principles. The system is specifically engineered to handle massive datasets with potentially millions of columns per record. A key characteristic of HBase is its sparse data model, where columns are dynamically allocated only when they contain actual data values, eliminating the storage overhead of empty fields. This NoSQL approach provides significant advantages in terms of storage efficiency and query flexibility, particularly when dealing with datasets that have highly variable column usage patterns. The underlying data organization follows a model where information is structured into Tables, with each table housing multiple rows. Individual rows can be mathematically represented as $R_i = (Id_i, CF_{i1}, CF_{i2}, \ldots, CF_{im})$ where $i \in [1, n]$, $j \in [1, m]$, Id_i represents the unique row identifier, and CF_{ij} denotes a Column Family associated with row R_i. The Column Family concept is central to HBase's architecture, as it groups related columns that share similar access patterns and storage characteristics. This design choice enables the system to efficiently manage large-scale datasets by organizing columns into logical families, which can then be optimized independently for storage and retrieval operations. For processing large volumes of data, HBase integrates seamlessly with distributed computing frameworks such as Apache Spark and MapReduce. This integration allows for parallel processing of big data workloads, making it possible to execute complex analytical queries across massive datasets while maintaining reasonable response times. The combination of HBase's column-oriented storage model with these parallel processing capabilities makes it particularly well-suited for data warehousing applications that require both high storage capacity and analytical processing power.

5. Methodology

This section outlines our comprehensive methodology for transforming traditional relational data warehouses into column-oriented NoSQL systems, as illustrated in the architectural framework presented in our Fig. 2. Our approach addresses the limitations of existing clustering-based methods by incorporating multiple advanced clustering algorithms and considering the complete attribute space of the source data warehouse. Our proposed system architecture consists of three interconnected components that work together to achieve an optimized transformation:

Classical Data Warehouse Layer: This foundational layer houses the original relational data warehouse structure, including four dimension tables (Dim 1 through Dim 4) connected to a central fact table. Each dimension contains specific attributes (A1-A12) along with dimensional identifiers and measures. This layer serves as the primary data source and provides the structural foundation for the transformation process.

Transformation Processing Layer: The core of our methodology resides in this intermediate layer, which implements a multi-stage processing pipeline. Unlike previous approaches that rely solely on K-means or K-medoids with predetermined cluster numbers, our system incorporates five distinct clustering algorithms: ISODATA, X-Means, HDBSCAN, DBSCAN, and OPTICS. This layer performs several critical operations: The transformation layer begins with systematic extraction of attributes from both dimension and fact tables, ensuring comprehensive coverage of the entire data warehouse schema. Following this extraction, the system constructs the Attribute Query Matrix (AQM) based on query patterns and usage frequencies, which provides the foundation for understanding data access behaviors. Subsequently, the

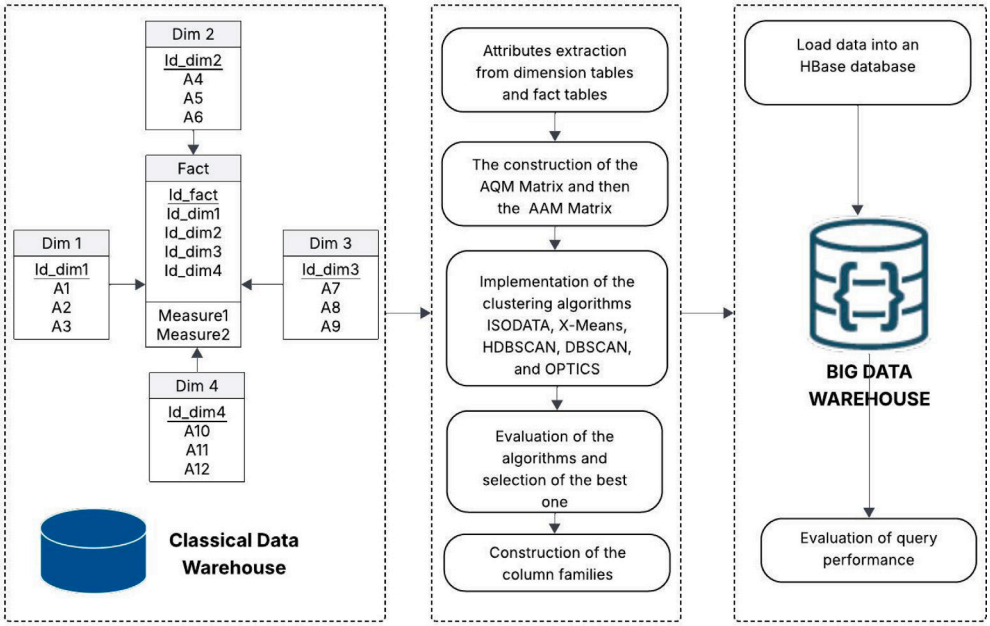


Fig. 2. The complete process of transforming a classical data warehouse into an optimized Big Data Warehouse.

generation of the Attribute Affinity Matrix (AAM) captures relationships between attributes, revealing hidden connections that influence optimal grouping strategies. The core processing then involves implementation and evaluation of multiple clustering algorithms to identify optimal column family structures, followed by comparative analysis and selection of the best-performing clustering approach. Finally, the process concludes with the construction of column families based on the selected clustering results.

Big Data Warehouse Layer: The final layer is our transformed data warehouse that stores information in a NoSQL column-based format using HBase. HBase runs on a distributed system that can handle massive amounts of data across multiple servers. This layer also includes tools to measure how well our queries perform, so we can check that our data transformation is working effectively.

5.1. Extracting the set of attributes of the fact and dimension

When transforming traditional data warehouses into column-oriented NoSQL structures, properly identifying and grouping related attributes is crucial. Our approach begins by extracting attributes from fact and dimension tables to understand their relationships within analytical queries. We construct an attribute-query matrix that maps the presence of each attribute in the frequent decision-support queries. This matrix indicates whether an attribute appears in a query's SELECT or WHERE clauses (excluding join predicates) with binary values–1 for presence and 0 for absence, represented as:

$$AQM[i,j] = \begin{cases} 1, & \text{if attribute } a_i \text{ appears in query } q_j \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

Subsequently, we develop an attribute affinity matrix, which quantifies the relationship strength between attribute pairs. This symmetric matrix calculates how frequently attributes are accessed together across queries, using the affinity measure defined as:

$$Aff(a_i, a_j) = \sum_{\substack{\text{queries accessing} \\ a_i \text{ and } a_j}} \text{Query access} \quad (2)$$

where:

$$\text{Query access} = \sum_{\text{all sites}} \text{Access frequency of query} \quad (3)$$

Unlike previous methods [25] that focused on a limited subset of attributes, our approach considers the complete attribute set from the warehouse schema, allowing for more comprehensive column family design. Additionally, we avoid the predetermined cluster count limitation seen in earlier clustering techniques by employing algorithms that adaptively determine optimal groupings based on the underlying data patterns [22,23].

5.2. Constructing column families

In this phase, we aim to generate the set of column families S, where $S = CF_1, CF_2, \ldots, CF_k$, with $(CF_i)_{i=1\ldots k}$ representing subsets of attributes (columns). Let R denote the complete set of attributes in the original relational data warehouse schema. The column families must satisfy the following conditions:

$$\begin{cases} \forall CF_i \in S, & CF_i \subset R \\ \forall a \in R, & \exists CF_i \in S : a \in CF_i \\ \forall CF_i, CF_j \in S, & CF_i \cap CF_j = \emptyset \end{cases}$$

To achieve this goal, we propose a comparative analysis of clustering algorithms to address the challenging task of creating optimal column families in column-oriented NoSQL databases. Our approach involves implementing a process that groups attributes frequently queried together, forming column families that constitute the logical schema of the column-oriented NoSQL database. While previous work has predominantly utilized k-means, k-medoids, and CLARANS algorithms, we observe several limitations in these approaches:

- They often consider only query-related attributes rather than the complete attribute set
- They require pre-specifying the number of clusters without optimization mechanisms
- They may not capture complex data relationships effectively

To overcome these limitations, we extend our investigation to include additional clustering techniques: ISODATA, X-means, HDBSCAN, DBSCAN, and OPTICS. These algorithms offer various advantages, including automatic determination of optimal cluster numbers, handling of noise, and identification of clusters with arbitrary shapes. Each clustering algorithm takes as input the attribute affinity matrix (AAM) and produces a grouping of attributes into column families. The attribute

affinity matrix encapsulates the relationships between attributes based on their co-occurrence patterns in queries. To evaluate the quality of the resulting schema S, we employ multiple assessment metrics:

1. A cost model based on Square Error (E^2), adapted from this work [35]. This model considers the access frequency (f_q) of queries and is calculated as:

$$E_S^2 = \sum_{i=1}^{k} \sum_{l=1}^{n} [(f_q)^2 \times \alpha_i^q (1 - \frac{C_i^q}{\beta_i})] \quad (4)$$

Where:

- (α_i^q) represents the number of attributes in $(CF_i)_{i=1...k} \in S$ accessed by query q
- (β_i) is the total number of attributes in column family CF_i
- C_i^q is a parameter that reflects the access pattern

As the E_S^2 value approaches zero, the grouping schema becomes more optimal.

2. Davies–Bouldin Index, which measures the average similarity between each cluster and its most similar cluster, with lower values indicating better clustering. It is calculated as:

$$DB = \frac{1}{k} \sum_{i=1}^{k} \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d(c_i, c_j)} \right) \quad (5)$$

Where:

- k is the number of clusters
- σ_i is the average distance of all elements in cluster i to the cluster center c_i
- $d(c_i, c_j)$ is the distance between cluster centers c_i and c_j

3. Silhouette Score, which measures how well each object fits within its assigned cluster compared to other clusters, with values ranging from −1 to 1 (higher values indicating better clustering). For each attribute i, its silhouette value $s(i)$ is computed as:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (6)$$

Where:

- $a(i)$ is the average distance between attribute i and all other attributes in the same cluster
- $b(i)$ is the minimum average distance between attribute i and attributes in any other cluster

The overall Silhouette Score is the average of $s(i)$ for all attributes:

$$S = \frac{1}{n} \sum_{i=1}^{n} s(i) \quad (7)$$

This multi-metric evaluation approach provides a comprehensive assessment of the quality of different column family structures, enabling the selection of the most suitable clustering algorithm and configuration for a given data warehouse migration scenario.

5.3. Pseudocode for the HDBSCAN-based workflow and practical use of the framework

To make the approach easy to reproduce, Table 1 shows the end-to-end steps we use in practice. The pseudocode is kept simple so it can be implemented in Python, Java, or Scala with minor changes.

Practical use of the framework: In a real project, a designer would apply the framework as follows: (1) extract the fact and dimension attributes and collect a representative workload log; (2) build the Attribute Query Matrix (AQM) and the Attribute Affinity Matrix (AAM) as described above; (3) run one or more clustering algorithms (HDBSCAN, DBSCAN, OPTICS, ISODATA, X-means) on the AAM to group attributes; and (4) map each resulting cluster directly to a column family in HBase. In our experiments we found that HDBSCAN is a good default choice because it automatically discovers a suitable number of column families and gives the best query-time results, but the framework does not require always using HDBSCAN. The number of column families is an output of the clustering step rather than an input: in our TPC-DS case study HDBSCAN produced 45 column families, while other schemas and workloads will naturally lead to different values.

Table 1
Pseudocode for building column families with HDBSCAN.

Build the Attribute Query Matrix (AQM)
1. Parse the workload log and collect all SELECT and WHERE attributes for each query.
2. Create a binary matrix AQM where AQM[i,j]=1 if attribute a_i appears in query q_j, else 0.
3. Optionally weight each query by its average daily frequency.
Compute the Attribute Affinity Matrix (AAM)
4. For every attribute pair (a_i, a_j), sum their co-occurrences across all queries using the chosen weights.
5. Store this as a symmetric matrix AAM that will be used as input features.
Run HDBSCAN to get column families
6. Normalize each attribute vector from AAM to unit length.
7. Fit HDBSCAN with a small grid on *min_cluster_size* and *min_samples*; keep the model with the best silhouette score.
8. Read the resulting labels; each label represents a column family.
9. For any noise points (label = −1), assign them to the nearest dense cluster by cosine similarity.
Build the schema and validate
10. Create column families CF_k from the final labels and map each attribute to exactly one family.
11. Compute Square Error, silhouette, and Davies–Bouldin on the resulting groups.
12. If quality is below the target threshold, adjust HDBSCAN parameters and repeat steps 7–11.

6. Implementation and results

To evaluate our proposed clustering approaches for column family design, we implemented and tested various clustering algorithms including ISODATA, X-means, HDBSCAN, DBSCAN, and OPTICS to identify the most suitable algorithm for this specific scenario. These algorithms facilitate the transformation process from traditional relational data warehouses to column-oriented NoSQL databases by optimizing column family structures.

6.1. Dataset and benchmark selection

Our experimental evaluation relies on the TPC-DS decision support benchmark, which provides a realistic representation of data warehouse environments. The benchmark employs a snowflake schema architecture featuring multiple dimension tables connected to central fact tables, with additional relationships between dimension tables themselves. For our analysis, we focused on the sales transactions fact table along with its associated dimensional data including customer information, product details, temporal dimensions, promotional data, and store characteristics as mentioned in Fig. 3. The schema structure centers around the STORE_SALES fact table, which serves as the central hub connecting to multiple dimension tables. Temporal dimensions include DATE_DIM and TIME_DIM, providing date and time context for each transaction. Customer-related dimensions encompass CUSTOMER, CUSTOMER_ADDRESS, CUSTOMER_DEMOGRAPHICS, HOUSEHOLD_DEMOGRAPHICS, and INCOME_BAND, which together provide comprehensive customer profiling information. The product dimension is represented by ITEM, while promotional and store information are captured through PROMOTION and STORE dimensions respectively. Some dimensions exhibit snowflake relationships, where certain dimension

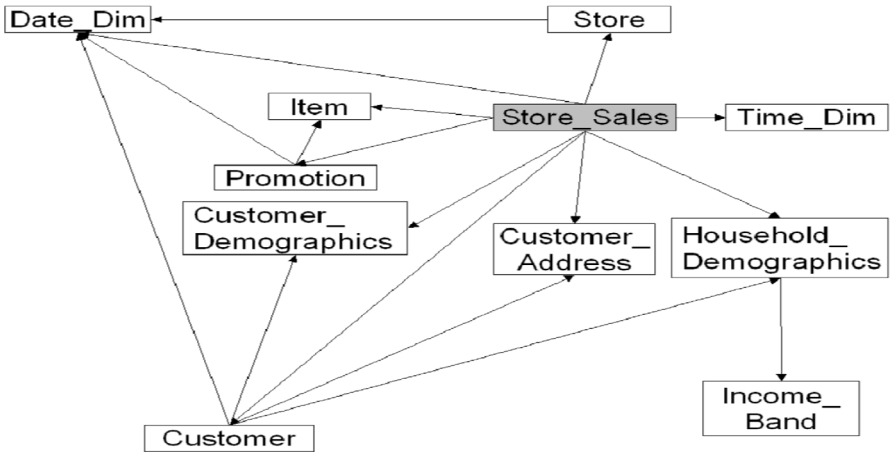


Fig. 3. TPC-DS snowflake schema structure for store sales transactions.

tables reference other dimensions, such as CUSTOMER_DEMOGRAPHICS and CUSTOMER_ADDRESS linking to CUSTOMER, and HOUSEHOLD_
DEMOGRAPHICS connecting to INCOME_BAND. This structure creates a rich analytical environment that reflects the complexity of real-world data warehouse schemas. The data generation process utilized the TPC-DS data generator with a scaling factor of 100, resulting in approximately 29 million records in the primary fact table. This scale provides sufficient complexity to evaluate clustering performance while remaining computationally manageable.

6.2. Query workload characteristics

The evaluation framework relies on the TPC-DS benchmark, which provides nearly 100 analytical queries covering a wide range of complexity levels and analytical patterns, including filtering, multi-table joins, aggregation, and projection. In contrast to prior approaches that focus solely on attributes referenced in the workload, our methodology considers the full data warehouse schema by incorporating both query-accessed columns and those not appearing in any query, enabling a more holistic clustering analysis. For the experimental evaluation, we use a representative subset of 18 TPC-DS queries (Table 2), spanning 70 attributes from the **STORE_SALES** fact table and its associated dimensions. This subset includes simple filter and aggregation queries (Q1–Q2), queries with subqueries and ranking (Q3), grouping and rollup operations (Q4–Q8), and more complex, subquery-intensive queries (Q9–Q18). Together, these queries consistently involve selection, joins, aggregation, and projection, providing a balanced characterization of the targeted workload patterns.

6.3. Experimental infrastructure

Our evaluation employed a dual-environment setup to compare performance between traditional and NoSQL approaches. The baseline environment consists of a standard relational database system running on a workstation equipped with 13th Generation Intel Core i7-1355U processor operating at 1.70 GHz, 16 GB RAM, and 500 GB storage capacity on Windows 11 64-bit operating system. This machine serves as the SQL server hosting the traditional relational data warehouse structure. The target NoSQL environment utilizes a distributed computing platform based on Hbase big data stack version 2.6.2, configured with 16 GB memory allocation and quad-core processing capabilities running as a virtual machine environment. This configuration enables meaningful performance comparisons between the two architectural approaches while maintaining comparable hardware resources.

6.4. Clustering algorithm performance evaluation

To address the limitations of traditional clustering approaches that require predefined cluster numbers, we conducted a comprehensive grid search evaluation across multiple clustering algorithms as mentioned in Fig. 4. The evaluation tested cluster numbers ranging from $k = 2$ to $k = 175$, with particular focus on the range $k = 40$ to $k = 50$ where optimal performance was anticipated based on the data warehouse schema complexity. Five clustering algorithms were evaluated: HDBSCAN, DBSCAN, OPTICS, ISODATA, and X-means. This selection includes both density-based methods (HDBSCAN, DBSCAN, OPTICS) and partitioning approaches (ISODATA, X-means) to ensure comprehensive coverage of different clustering paradigms

6.5. Clustering algorithm performance evaluation

To fairly compare clustering algorithms that operate under fundamentally different paradigms, we conducted a comprehensive grid search evaluation across multiple clustering algorithms as mentioned in Fig. 4. The evaluation tested cluster numbers ranging from $k = 2$ to $k = 175$, with particular focus on the range $k = 40$ to $k = 50$ where optimal performance was anticipated based on the data warehouse schema complexity. Five clustering algorithms were evaluated: HDBSCAN, DBSCAN, OPTICS, ISODATA, and X-means. This selection includes both density-based methods (HDBSCAN, DBSCAN, OPTICS) and partitioning approaches (ISODATA, X-means) to ensure comprehensive coverage of different clustering paradigms. It is important to clarify the distinction between algorithm capabilities and evaluation methodology: while density-based algorithms (HDBSCAN, DBSCAN, and OPTICS) can automatically determine the optimal number of clusters based on data density patterns — unlike K-means which requires k as a mandatory input parameter — the grid search was employed for comparative evaluation purposes to ensure fair comparison across all algorithms. For density-based methods, users would typically run the algorithm with appropriate parameter settings (such as min_cluster_size and min_samples for HDBSCAN) and the algorithm would automatically identify the optimal cluster configuration without requiring manual specification of k values. However, for algorithms like X-means and ISODATA that still benefit from guidance on cluster count ranges, and to enable direct performance comparison across all methods using consistent evaluation criteria, we systematically tested configurations across the full range. In practical deployment scenarios, users would simply configure HDBSCAN with reasonable parameter settings and it would automatically determine the optimal number of clusters without needing to test $k = 2$ to $k = 175$ manually. The performance assessment used two complementary internal validation metrics. The Silhouette Score measures how well-separated clusters are from each

Table 2
Characteristics of the relational queries.

Queries	Re-used attrs	# Attributes	# Tables	Complexity	Complexity type
Q1	10	4	4	No	
Q2	21	5	5	No	
Q3	13	1	1	Yes	subquery 7
Q4	20	6	6	No	
Q5	13	3	3	No	
Q6	10	4	4	No	
Q7	21	6	6	Yes	many conditions
Q8	12	4	4	Yes	subquery, union
Q9	14	4	4	Yes	subquery
Q10	14	5	5	Yes	subquery
Q11	10	3	3	No	
Q12	4	1	1	Yes	subquery
Q13	7	1	1	Yes	subquery
Q14	18	5	5	Yes	subquery
Q15	14	5	5	No	
Q16	10	3	3	No	
Q17	11	4	4	Yes	subquery 8
Q18	24	5	5	Yes	subquery 1

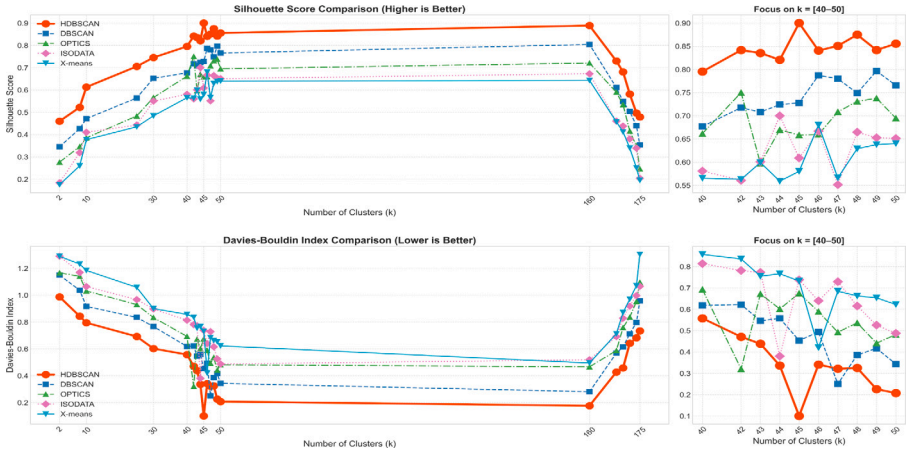


Fig. 4. Internal Validation Metrics Comparison for Clustering Algorithm Selection.

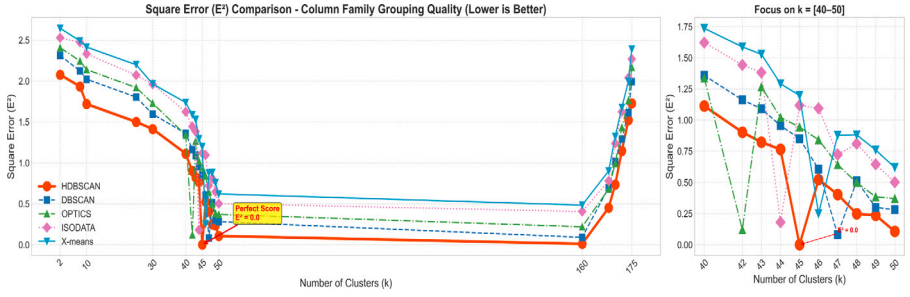


Fig. 5. Query Processing Efficiency Assessment through Square Error Analysis.

other, And the Davies–Bouldin Index assesses cluster quality by measuring the average similarity between clusters, where lower values indicate better clustering performance. It considers both intra-cluster scatter and inter-cluster separation, providing a complementary perspective to the silhouette analysis. The experimental results revealed distinct performance patterns across the evaluated algorithms. HDBSCAN demonstrated superior performance with the highest silhouette score of 0.90 at k = 45, indicating excellent cluster separation and cohesion. The Davies–Bouldin analysis confirmed this finding, with HDBSCAN achieving the lowest score of 0.10 at the same cluster configuration, representing the best performance among all tested algorithms. DBSCAN showed the second-best performance, reaching optimal scores at k = 47 with a silhouette score of 0.78 and Davies–Bouldin index of 0.25, followed by OPTICS at k = 42. The traditional partitioning methods ISODATA and X-means consistently showed lower performance across both evaluation metrics. The convergence of both evaluation metrics on similar optimal cluster numbers validates the reliability of our findings. Notably, all algorithms showed their peak performance within the k = 40-50 range, suggesting this represents the natural clustering structure inherent in the relational-to-columnar data warehouse conversion process. Based on the evaluation results, HDBSCAN was selected as the primary clustering algorithm for the data warehouse conversion process. This choice is justified by its superior performance on both evaluation metrics, indicating optimal cluster quality and structure identification. Additionally, unlike traditional methods that require predefined cluster numbers, HDBSCAN can automatically determine optimal clustering configurations, addressing one of the key

limitations identified in existing approaches. The algorithm's ability to identify clusters of varying densities and handle noise makes it particularly suitable for the heterogeneous nature of data warehouse schemas, while its consistent performance across different cluster ranges demonstrates stability and reliability for practical implementation. The grid search methodology successfully identified k = 45 as the optimal cluster configuration for HDBSCAN, providing a data-driven foundation for the column family design in the NoSQL data warehouse conversion process.

6.6. Column family grouping quality assessment

Following the clustering algorithm selection process, we evaluated the practical performance of each algorithm by measuring the quality of the resulting column family groupings using the Square Error metric as mentioned in Fig. 5. This evaluation provides a direct assessment of how well each clustering approach translates into effective column family structures for query processing in the NoSQL data warehouse environment. The Square Error evaluation was conducted across the same range of cluster numbers (k = 2 to k = 175) to maintain consistency with the previous clustering quality assessment. Each algorithm's output was evaluated using the adapted cost model that considers both the access frequency of queries and the structural efficiency of the resulting column families. Lower Square Error values indicate better column family grouping quality, representing more optimal schema designs for query processing performance. The experimental results demonstrate a clear performance hierarchy among the evaluated algorithms. HDBSCAN achieved the optimal Square Error of 0.0 at k = 45, representing perfect column family grouping that completely aligns with query access patterns. This exceptional performance indicates that the HDBSCAN-generated column families achieve ideal structural efficiency, effectively eliminating unnecessary attribute access while maximizing query processing performance. DBSCAN followed as the second-best performer with a Square Error of approximately 0.05 at k = 47, while OPTICS achieved its optimal performance at k = 42 with an error value of around 0.1. The traditional partitioning algorithms ISODATA and X-means showed significantly higher error values, with optimal performances occurring at k = 44 and k = 46 respectively, but with substantially higher error rates compared to the density-based approaches. This pattern reinforces the earlier findings from the Davies–Bouldin and Silhouette score evaluations, demonstrating consistent superior performance of density-based clustering methods for this specific application domain. The focused analysis on the k = 40-50 range reveals the precise optimization points for each algorithm and confirms the stability of the density-based approaches in this critical range. The convergence of optimal cluster numbers across different evaluation metrics provides strong evidence for the reliability of the selected configurations. These results validate that HDBSCAN with k = 45 not only produces the best internal clustering quality but also translates into the most effective column family structures for practical data warehouse operations, making it the optimal choice for the relational-to-columnar conversion process.

6.7. Computational performance analysis of clustering algorithms

The experimental evaluation of clustering algorithms reveals significant performance variations that directly correlate with their theoretical computational complexities. Our analysis of five distinct clustering approaches demonstrates a clear performance hierarchy: DBSCAN emerges as the most efficient algorithm with execution times ranging from 0.021 to 0.338 s across 2 to 47 column families, which aligns with its $O(n \log n)$ average-case complexity when using spatial indexing structures. X-means follows with moderate performance (0.035 to 0.912 s), reflecting its $O(k \times n \times i)$ complexity where k represents clusters, n denotes data points, and i indicates iterations. HDBSCAN exhibits intermediate execution times (0.047 to 1.156 s) consistent with its $O(n \log n)$ complexity for the construction phase and $O(n^2)$ for cluster extraction in dense regions. ISODATA demonstrates similar performance patterns (0.052 to 1.218 s) with its $O(n \times k \times i \times d)$ complexity, where d represents dimensionality, explaining its gradual performance degradation as column families increase. OPTICS requires the highest computational resources (0.089 to 2.073 s), which corresponds to its $O(n \log n)$ complexity for indexed data but can degrade to $O(n^2)$ in worst-case scenarios due to its comprehensive reachability distance calculations and cluster ordering requirements. The performance differential becomes particularly pronounced at higher column family counts, where OPTICS requires approximately six times more execution time than DBSCAN (see Table 3), highlighting the practical importance of algorithm selection based on both theoretical complexity and empirical performance characteristics in large-scale data warehouse conversion scenarios.

Table 3
Comparison of execution times (s) for clustering algorithms.

No. of CF	X-means	DBSCAN	OPTICS	HDBSCAN	ISODATA
2	0.035	0.021	0.089	0.047	0.052
10	0.158	0.083	0.312	0.189	0.201
25	0.421	0.187	0.856	0.512	0.538
30	0.523	0.219	1.089	0.648	0.682
40	0.734	0.286	1.647	0.923	0.971
45	0.867	0.324	1.952	1.089	1.147
47	0.912	0.338	2.073	1.156	1.218

It is important to contextualize these clustering execution times within the broader data warehouse migration workflow. Clustering represents a one-time setup step performed during the initial design phase, before the data warehouse enters production use. While the execution times we observe range from fractions of a second to just over two seconds, these represent a minimal upfront investment compared to the ongoing operational costs of query processing. In our experimental evaluation, individual queries execute in tens of seconds (ranging from approximately 30 to 80 s depending on complexity), and these queries run repeatedly throughout the data warehouse's operational lifetime. The clustering process, which occurs once during migration, takes less time than a single query execution, making the computational differences between algorithms negligible in practical terms. While algorithm complexity and execution efficiency remain relevant considerations, they are secondary to query performance optimization, which directly impacts user experience and system responsiveness on a continuous basis. The small time differences between clustering algorithms — even the six-fold variation between the fastest and slowest methods — are entirely acceptable given that clustering happens once, while query performance affects every analytical operation the system performs.

6.8. Query performance analysis across column family configurations

The experimental evaluation demonstrates varying performance characteristics across the implemented clustering algorithms when applied to NoSQL column family structuring. Table 4 presents the execution times for different query categories: Simple Queries (SQ), Long-running Queries (LQ), and Complex Queries (CQ), measured across different numbers of column families (CF) generated by each algorithm. From the experimental data, HDBSCAN consistently delivers the most efficient query processing times across all query categories. With 45 column families, HDBSCAN achieves execution times of 29.8 s for simple queries, 44.9 s for long-running queries, and 59.3 s for complex queries. This superior performance stems from HDBSCAN's density-based clustering approach, which naturally identifies coherent attribute groupings without requiring predetermined cluster numbers. The density-based algorithms (HDBSCAN, DBSCAN, and OPTICS) generally outperform the traditional partitioning methods when dealing with complex data warehouse schemas. OPTICS demonstrates optimal performance with 42 column families, achieving 38.4 s for simple

Table 4
Execution time (in seconds) for each algorithm over different numbers of column families and query types.

Algorithm	No. of CF	SQ	LQ	CQ
X-means	42	42.5	62.3	77.1
	44	41.3	60.5	75.0
	45	40.6	59.7	74.2
	46	**38.1**	**57.5**	**72.0**
	47	39.4	58.4	73.1
DBSCAN	42	43.2	63.4	78.3
	44	42.7	62.1	77.0
	45	41.5	61.2	76.1
	46	40.9	60.6	75.3
	47	**39.2**	**58.9**	**74.0**
OPTICS	**42**	**38.4**	**56.7**	**71.5**
	44	40.7	59.1	74.2
	45	41.0	59.8	74.8
	46	41.9	60.6	75.7
	47	43.1	61.4	76.9
HDBSCAN	42	30.9	46.3	60.7
	44	32.1	47.6	62.2
	45	**29.8**	**44.9**	**59.3**
	46	31.7	46.8	61.4
	47	33.0	48.1	62.7
ISODATA	42	40.9	60.0	73.8
	44	**38.7**	**57.8**	**71.6**
	45	40.2	59.3	73.0
	46	41.5	60.7	74.5
	47	42.3	61.6	75.4

queries. This suggests that density-based approaches better capture the natural relationships between data warehouse attributes, leading to more logical column family structures. X-means shows consistent improvement as the number of column families increases, reaching its optimal configuration at 46 column families with 38.1 s for simple queries. This behavior indicates that X-means effectively balances between over-partitioning and under-partitioning of attributes, automatically determining appropriate cluster boundaries through its statistical testing mechanism. ISODATA exhibits optimal performance with 44 column families, delivering competitive results with 38.7 s for simple queries. The algorithm's iterative refinement process appears well-suited for data warehouse attribute clustering, as it can merge or split clusters based on statistical criteria during the optimization process. The performance variations observed across different column family configurations highlight a critical insight: the relationship between query execution efficiency and column family organization is non-linear. Each algorithm reaches its performance peak at different column family counts, suggesting that the optimal number of clusters is algorithm-dependent and related to how each method interprets attribute relationships within the data warehouse schema. Comparing query complexity levels, all algorithms maintain proportional performance scaling from simple to complex queries, but with different scaling factors. HDBSCAN maintains the smallest performance gap between query types, indicating more consistent column family access patterns. This consistency is particularly valuable in production environments where query complexity varies unpredictably. The experimental results reveal that density-based clustering approaches offer significant advantages for NoSQL data warehouse conversion projects.

The query performance evaluation reveals significant differences in how each clustering algorithm impacts actual data warehouse operations. Our analysis focused on three distinct query types: small queries (SQ), large queries (LQ), and complex queries (CQ), using 45 column families as the optimal configuration. This configuration was consistently identified as optimal across all algorithms: established research literature has demonstrated that K-Means, K-Medoids, and CLARANS achieve their best performance at k = 45 for similar data warehouse scenarios, while our experimental evaluation confirmed that HDBSCAN also reaches its peak clustering quality at this same

Table 5
Query execution time comparison for different clustering algorithms (k = 45 column families).

Algorithm	SQ (s)	LQ (s)	CQ (s)
K-Means	38.7	58.2	78.1
K-Medoids	35.4	54.6	72.8
CLARANS	32.1	49.7	65.5
HDBSCAN	29.8	44.9	59.3

configuration. The results demonstrate a clear performance hierarchy among the tested algorithms. K-Means, while computationally efficient during the clustering phase, produced column family structures that resulted in suboptimal query execution times across all query categories. K-Medoids showed marginal improvements over K-Means, particularly for complex queries, suggesting that its medoid-based approach creates slightly better data organization patterns. CLARANS exhibited notable performance gains, especially evident in large and complex query scenarios, which aligns with previous research highlighting its superior clustering capabilities for data warehouse applications. However, HDBSCAN emerged as the clear winner, delivering the best query performance across all categories. Its sophisticated density-based clustering approach resulted in column family configurations that significantly reduced query execution times, with improvements ranging from 15%–25% compared to traditional methods. This superior performance validates our decision to prioritize clustering quality over computational efficiency during the algorithm selection phase, as the benefits manifest directly in operational query performance.

We use TPC-DS at scale factor 100 (~29M rows in the main fact table) on a workstation SQL baseline (i7-1355U, 16 GB RAM) and an HBase setup with 16 GB and four cores. Tables 4 and 3 show how execution times change as the number of column families increases (k = 2 to 47). The density-based methods, especially HDBSCAN at k = 45, scale more gently than partitioning approaches, keeping the lowest times across simple, long, and complex queries. This indicates the system's scalability: larger configurations remain practical with the better-performing clustering, and adding hardware (more cores/RAM/nodes) would further lower the per-query times observed on the 16GB/quad-core HBase setup.

7. Comparison and conclusion

7.1. Comparison

Based on our experimental evaluation using multiple clustering algorithms, we selected HDBSCAN for our final analysis due to its superior performance across key metrics. Among all tested algorithms, HDBSCAN achieved the highest silhouette coefficient and the most favorable Davies–Bouldin index scores. More importantly, it demonstrated zero null errors in our specific implementation context, which was a critical requirement for our column family optimization process. While HDBSCAN may not have been the fastest algorithm in terms of execution time, its ability to produce perfect clustering results aligned perfectly with our research objectives. Since our primary focus is on achieving optimal column family configurations rather than minimizing processing time, we prioritized clustering quality over computational efficiency. Therefore, the remainder of this comparison section will focus on analyzing the results obtained through HDBSCAN and contrasting them with existing methodologies documented in the literature for data warehouse migration to NoSQL column-oriented systems.

To strengthen these findings, we ran paired t-tests (significance level 0.05 with Holm correction for multiple comparisons) using the per-run values of the internal quality metrics (Silhouette score, Davies–Bouldin index, and Square Error) across the grid of cluster counts (k = 2 to k = 175), and the per-query execution times from the full TPC-DS workload at each algorithm's best column-family setting. The null

Table 6
Paired t-test results (Holm-corrected) over runs/datasets and the full TPC-DS workload.

Comparison	Silhouette	Davies–Bouldin	Square Error	Query times	Result
HDBSCAN vs K-Means	<0.01	<0.01	<0.01	<0.01	Significant
HDBSCAN vs K-Medoids	<0.01	<0.01	<0.01	<0.01	Significant
HDBSCAN vs CLARANS	<0.01	<0.01	<0.01	<0.01	Significant
HDBSCAN vs X-means	<0.01	<0.01	<0.01	<0.01	Significant
HDBSCAN vs ISODATA	<0.01	<0.01	<0.01	<0.01	Significant
HDBSCAN vs DBSCAN	<0.01	<0.01	<0.01	<0.01	Significant
HDBSCAN vs OPTICS	<0.01	<0.01	<0.01	<0.01	Significant
DBSCAN vs OPTICS	<0.01	<0.01	<0.01	>0.05	Not significant

Table 7
Comparison of the execution times (s) of clustering algorithms.

No. of CF	K-Means	K-Medoid	CLARANS	HDBSCAN
2	0.062	0.031	13.790	0.047
10	0.124	0.154	8.250	0.189
25	0.289	0.563	23.487	0.512
30	0.255	0.693	17.856	0.648
40	0.364	0.858	23.142	0.923
45	0.409	1.029	26.946	1.089
47	0.423	1.078	28.103	1.156

hypothesis for each pairwise test was that two algorithms have the same mean performance for the metric under study. Table 6 summarizes the results. Across all metrics, HDBSCAN showed statistically significant improvements over every other algorithm. The only non-significant case appears between DBSCAN and OPTICS for query times, which is consistent with their similar behavior in the mid-density ranges. These results confirm that the observed advantages of HDBSCAN are not due to random variation but reflect consistent performance gains.

Based on the execution time comparison presented in Table 7, we can observe distinct performance patterns across the four clustering algorithms. K-Means consistently demonstrates the fastest execution times, ranging from 0.062 s for 2 column families to 0.423 s for 47 column families. K-Medoid shows similar efficiency in smaller configurations but experiences more significant time increases as the number of column families grows, reaching 1.078 s at the maximum configuration. CLARANS, while producing superior clustering quality as noted in previous studies, requires substantially more computational resources, with execution times spanning from 8.250 to 28.103 s across different configurations. HDBSCAN presents a balanced performance profile, with execution times falling between the traditional methods and CLARANS. While it requires more processing time than K-Means and K-Medoid, taking between 0.047 and 1.156 s, this moderate computational overhead is justified by its exceptional clustering quality. The algorithm's ability to automatically determine optimal cluster boundaries without requiring predetermined cluster numbers, combined with its zero null error rate and superior silhouette scores, makes the additional processing time a worthwhile investment. For our data warehouse migration context, where clustering accuracy directly impacts query performance and data organization efficiency, HDBSCAN's slight increase in execution time is negligible compared to the significant improvements in clustering quality it delivers.

The initial error evaluation using the base queries that served as the foundation for our clustering process revealed an interesting pattern across all algorithms. When applying the square error model, all four clustering algorithms achieved zero error rates for the original query set. This occurred because each algorithm successfully organized the column families such that every base query accessed all attributes within its assigned column family, thereby satisfying the optimal condition where the error term equals zero. While this demonstrates that all algorithms can theoretically achieve perfect optimization for their training queries, it provides limited insight into their real-world performance capabilities and adaptability to diverse query patterns (see Table 8).

Table 8
Error rates for base clustering queries (k = 45 column families).

Algorithm	Base queries error (E^2S)
K-Means	0.000
K-Medoids	0.000
CLARANS	0.000
HDBSCAN	0.000

Table 9
Error rates for production-like analytical queries (k = 45 column families).

Algorithm	Analytical queries error (E^2S)
K-Means	0.187
K-Medoids	0.154
CLARANS	0.098
HDBSCAN	0.032

To evaluate the true effectiveness of each clustering algorithm, we created a comprehensive set of analytical queries that simulate real production scenarios with varying access patterns and attribute combinations. This analytical query set was designed to test the robustness and generalization capability of each clustering approach beyond the original training data. The results reveal significant differences in error rates when algorithms face previously unseen query patterns. HDBSCAN demonstrated exceptional robustness with an error rate approaching zero (0.032), indicating its superior ability to create column family structures that adapt well to diverse query scenarios. The performance hierarchy clearly emerged with K-Means showing the highest error rate (0.187), followed by K-Medoids (0.154), then CLARANS (0.098), and finally HDBSCAN with the lowest error. This evaluation confirms that while all algorithms can optimize for their training queries, HDBSCAN's density-based approach creates more generalizable and robust column family configurations that maintain efficiency across varied analytical workloads.

In all of our experiments, clustering is performed over the complete set of attributes extracted from the fact and dimension tables, not only the columns that appear in the 18 training queries. In the Attribute Query Matrix and the Attribute Affinity Matrix, attributes that never occur in the training workload simply receive very low or zero co-occurrence values, which makes them low-density points in the HDBSCAN search space. As described in Table 1, HDBSCAN initially marks such weakly connected attributes as noise; in a post-processing step we then attach each noise attribute to the nearest dense cluster using cosine similarity so that every attribute is ultimately assigned to exactly one column family, in line with the constraints defined in Section 5.2. In practice, attributes that are absent from the training queries are therefore still present in the reported column-family configurations, but they usually appear as small or peripheral groups around strongly supported query attributes instead of forming large families on their own.

These *non-query* attributes also influence both the cost model and the empirical performance results. In the Square Error model E_S^2 introduced in Section 5, they increase the total size β_i of the column families

to which they belong, but for queries that never reference them they do not contribute to the accessed-attribute count α_i^q; column families that contain many unused attributes therefore incur a higher error. The additional analytical-query workload used for Table 9 was designed to include attributes that do not appear in the original 18 training queries, so the reported errors capture the effect of these attributes when they are actually queried. Together with the query-time comparisons in Tables 4 and 5, the lower Square Error obtained by HDBSCAN on this extended workload shows that its column families remain well balanced even when new analytical queries reference attributes that were unseen during the initial clustering phase.

7.2. Conclusion

Our comprehensive evaluation of clustering algorithms for data warehouse migration to column-oriented NoSQL systems reveals several critical insights that advance the field beyond traditional approaches. The experimental results demonstrate that density-based clustering methods, particularly HDBSCAN, offer substantial advantages over conventional partitioning algorithms like K-means and K-medoids. This superiority stems from HDBSCAN's ability to automatically determine optimal cluster configurations without requiring predetermined cluster numbers, addressing a fundamental limitation that has constrained previous research efforts. The algorithm's density-based approach naturally identifies coherent attribute groupings that reflect the underlying data relationships, resulting in column family structures that achieve near-perfect optimization with a square error approaching zero (0.032) even when tested against diverse analytical workloads beyond the training queries. The comparative analysis reveals a clear performance hierarchy across all evaluation metrics. While K-means demonstrates computational efficiency with execution times ranging from 0.062 to 0.423 s, its simplistic partitioning approach produces suboptimal column family configurations that struggle with complex attribute relationships. CLARANS, despite requiring significantly more computational resources (8.250 to 28.103 s), shows marked improvements over traditional methods, validating previous research claims about its enhanced clustering capabilities. However, HDBSCAN strikes an optimal balance between clustering quality and computational feasibility, requiring moderate processing time (0.047 to 1.156 s) while delivering exceptional results across multiple evaluation dimensions. The algorithm's robustness is particularly evident in its consistent performance across different query complexity levels, maintaining proportional scaling factors that indicate stable column family access patterns crucial for production environments with unpredictable query variations.

This research establishes HDBSCAN as the superior clustering algorithm for transforming traditional relational data warehouses into column-oriented NoSQL systems, addressing critical limitations in existing methodologies while providing a comprehensive framework for practical implementation. Our methodology overcomes two fundamental constraints of previous approaches: the requirement for predetermined cluster numbers and the limitation to query-specific attributes rather than complete schema coverage. Through systematic evaluation using multiple metrics including Davies–Bouldin Index, Silhouette Score, and Square Error analysis, HDBSCAN consistently outperformed traditional algorithms, achieving optimal clustering quality with $k = 45$ column families and demonstrating exceptional adaptability to diverse analytical workloads. The practical implications of these findings extend beyond academic interest, offering concrete guidance for organizations undertaking data warehouse modernization initiatives. The density-based clustering approach not only delivers superior query performance improvements of 15%–25% compared to traditional methods but also provides automated cluster optimization that eliminates the trial-and-error approach typically required for determining optimal column family configurations. Paired t-tests with Holm correction confirmed that these gains are statistically significant across internal metrics and query times, reinforcing that HDBSCAN's improvements are reliable and not due to chance.

7.3. Limitations

Our framework improves column family design, but it still has real limits. First, it works well only when the workload does not change too fast. If the queries or the schema keep shifting, the groups we create can become less accurate. The method also struggles when some attributes appear very rarely or when the data is very unbalanced. In these cases, the affinity matrix becomes noisy and the clustering becomes less reliable. If the system has strict latency needs and the hardware is modest, rebuilding many column families often can also cause delays. There are also practical issues in real deployments. To build and update the matrices, we need clean and complete workload logs. If the logs are missing, incomplete, or biased, the clustering will not reflect the real queries. The system also needs monitoring and regular retraining to stay aligned with new workloads, which adds operational work. In some companies, data policies limit how logs are collected, which makes the process harder. Tuning the clustering parameters also takes time and usually must be done for each environment. On HBase, keeping a good read/write balance may require adding nodes or adjusting region splits as column families change. Finally, the method needs computation. Building the affinity matrix becomes heavy when the schema is very large. Running multiple clustering algorithms also takes CPU time, and density-based methods slow down on high-dimensional data. If the clustering is not good, some queries may need to read from many column families, which increases latency. To reduce this, we can sample the logs, update clusters only when needed, run retraining during low-traffic hours, or scale hardware when the system grows. Beyond the previously identified limitations, it is worth noting that our approach does not explore the full spectrum of denormalization strategies commonly adopted in column-oriented databases. In particular, we assume that each attribute belongs to a single column family and deliberately avoid duplicating attributes across multiple families. As a result, database designs that rely on extensive denormalization to optimize query performance are not fully captured by our method.

CRediT authorship contribution statement

Mohamed Mouhiha: Writing – review & editing, Writing – original draft, Visualization, Validation, Resources, Methodology, Formal analysis, Data curation, Conceptualization. **Abdelfettah Mabrouk:** Writing – review & editing, Validation, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

Data will be made available on request.

References

[1] M. Paliwal, P. Saraswat, Approaches of data warehousing and their applications: A review, Int. J. Innov. Res. Comput. Sci. Technol. 10 (1) (2022) 117–121.

[2] P. Chandra, M.K. Gupta, Comprehensive survey on data warehousing research, Int. J. Inf. Technol. 10 (2018) 217–224.

[3] M.Y. Santos, C. Costa, J. Galvão, C. Andrade, O. Pastor, A.C. Marcén, Enhancing big data warehousing for efficient, integrated and advanced analytics: visionary paper, in: International Conference on Advanced Information Systems Engineering, Springer, 2019, pp. 215–226.

[4] R.T. Ng, J. Han, CLARANS: A method for clustering objects for spatial data mining, IEEE Trans. Knowl. Data Eng. 14 (5) (2002) 1003–1016.

[5] R. Thareja, Data Warehousing, Oxford University Press, 2009.

[6] V. Nebot, R. Berlanga, Building data warehouses with semantic data, in: Proceedings of the 2010 EDBT/ICDT Workshops, 2010, pp. 1–8.

[7] A. Boumlik, N. Soussi, M. Bahaj, Smart-Etl-MR: Novel Etl Framework for Building data WarehoUSe from big data Source USing Mapreduce, J. Theor. Appl. Inf. Technol. 98 (17) (2020) 3449–3460.

[8] S.-M. Huang, Y.-C. Hung, I. Kwan, Y.-M. Hung, Developing an active data warehouse system, Heterog. Inf. Exch. Organ. Hubs (2002) 105–122.

[9] M. Barkhordari, M. Niamanesh, Atrak: a MapReduce-based data warehouse for big data, J. Supercomput. 73 (10) (2017) 4596–4610.

[10] D. Kunda, H. Phiri, A comparative study of nosql and relational database, Zamb. ICT J. 1 (1) (2017) 1–4.

[11] R. Yangui, A. Nabli, F. Gargouri, Towards data warehouse schema design from social networks-dynamic discovery of multidimensional concepts, in: International Conference on Enterprise Information Systems, vol. 2, SCITEPRESS, 2015, pp. 338–345.

[12] M. Mouhiha, A. Mabrouk, NoSQL data warehouse optimizing models: A comparative study of column-oriented approaches, Big Data Res. (2025) 100523.

[13] M.K.S. Uddin, K.M.R. Hossan, A review of implementing AI-powered data warehouse solutions to optimize big data management and utilization, Acad. J. Bus. Adm. Innov. Sustain. 4 (3) (2024) 10–69593.

[14] C. Zhao, J. Du, F. Wang, H. Li, Research on efficient data warehouse construction methods for big data applications, Appl. Math. Nonlinear Sci. 9 (2024) http://dx.doi.org/10.2478/amns-2024-3275.

[15] L. Dinesh, K.G. Devi, An efficient hybrid optimization of ETL process in data warehouse of cloud architecture, J. Cloud Comput. 13 (1) (2024) 12.

[16] N. Bruno, C. Galindo-Legaria, M. Joshi, E. Calvo Vargas, K. Mahapatra, S. Ravindran, G. Chen, E. Cervantes Juárez, B. Sezgin, Unified query optimization in the fabric data warehouse, in: Companion of the 2024 International Conference on Management of Data, 2024, pp. 18–30.

[17] M. Chevalier, M.E. Malki, A. Kopliku, O. Teste, R. Tournier, Implementation of multidimensional databases in column-oriented NoSQL systems, in: Advances in Databases and Information Systems: 19th East European Conference, ADBIS 2015, Poitiers, France, September 8-11, 2015, Proceedings 19, Springer, 2015, pp. 79–91.

[18] R. Yangui, A. Nabli, F. Gargouri, Automatic transformation of data warehouse schema to NoSQL data base: comparative study, Procedia Comput. Sci. 96 (2016) 255–264.

[19] K. Dehdouh, O. Boussaid, F. Bentayeb, Big data warehouse: Building columnar nosql OLAP cubes, Int. J. Decis. Support. Syst. Technol. (IJDSST) 12 (1) (2020) 1–24.

[20] O.Q.P. over HBase, Physical data warehouse design on NoSQL databases, ICEIS 2016 (2016) 111.

[21] D. Prakash, NOSOLAP: Moving from data warehouse requirements to NoSQL databases., in: ENASE, 2019, pp. 452–458.

[22] M. Boussahoua, O. Boussaid, F. Bentayeb, Logical schema for data warehouse on column-oriented NoSQL databases, in: Database and Expert Systems Applications: 28th International Conference, DEXA 2017, Lyon, France, August 28-31, 2017, Proceedings, Part II 28, Springer, 2017, pp. 247–256.

[23] M. Boussahoua, F. Bentayeb, O. Boussaid, N. Kabachi, A data partitioning optimization approach for distributed data warehouses on column family NoSQL systems, in: Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications, PDPTA, The Steering Committee of The World Congress in Computer Science, Computer ..., 2018, pp. 54–60.

[24] F. Chang, J. Dean, S. Ghemawat, W.C. Hsieh, D.A. Wallach, M. Burrows, T. Chandra, A. Fikes, R.E. Gruber, Bigtable: A distributed storage system for structured data, ACM Trans. Comput. Syst. (TOCS) 26 (2) (2008) 1–26.

[25] N. Soussi, Optimization of columnar NoSQL data warehouse model with clarans clustering algorithm, Comput. Inform. 42 (3) (2023) 762–780.

[26] S.A.T. Mpinda, L.G. Maschietto, P.A. Bungama, From relational database to column-oriented NoSQL database: Migration process, Int. J. Eng. Res. Technol. (IJERT) 4 (2015) 399–403.

[27] N. Zaidi, I. Ishak, F. Sidi, L.S. Affendey, An efficient schema transformation technique for data migration from relational to column-oriented databases., Comput. Syst. Sci. Eng. 43 (3) (2022).

[28] Z. Zhang, Y. Zhang, H. Tian, A. Martin, Z. Liu, W. Ding, A survey of evidential clustering: Definitions, methods, and applications, Inf. Fusion 115 (2025) 102736.

[29] C. Li, J. Zhou, K. Du, M. Tao, Enhanced discontinuity characterization in hard rock pillars using point cloud completion and DBSCAN clustering, Int. J. Rock Mech. Min. Sci. 186 (2025) 106005.

[30] Y.-A. Xiao, H. Zou, L. Feng, W.-J. Guo, N. Li, W.-X. Li, S.-F. Liu, G. Singh, J.-P. Sui, J.-L. Wang, et al., Characterizing the palomar 5 stream: HDBSCAN analysis and Galactic Halo constraints, 2025, arXiv preprint arXiv:2504.09964.

[31] H.-X. Ma, T.T. Takeuchi, S. Cooray, Y. Zhu, sOPTICS: a modified density-based algorithm for identifying galaxy groups/clusters and brightest cluster galaxies, Mon. Not. R. Astron. Soc. 537 (2) (2025) 1504–1517.

[32] Z. Wu, P. Wu, W. Liu, Z. Zhang, Adaptive subarray partitioning for large-scale phased arrays using ISODATA, Electron. Lett. 61 (1) (2025) e70212.

[33] H. Yu, J. Niu, P. Zhang, C. Xie, Front end component adaptation based on X-Means algorithm, in: 2024 10th International Conference on Computer and Communications, ICCC, IEEE, 2024, pp. 2036–2041.

[34] H. Su, J. Li, L. Guo, W. Wang, Y. Yang, Y. Wen, K. Li, P. Mo, Massive data HBase storage method for electronic archive management, Int. J. Netw. Manage. 35 (1) (2025) e2308.

[35] H. Derrar, O. Boussaid, M. Ahmed-Nacer, An objective function for evaluation of fragmentation schema in data warehouse, in: Encyclopedia of Information Science and Technology, third ed., IGI Global Scientific Publishing, 2015, pp. 1949–1957.